LAPORAN TUGAS INDIVIDU I-4

ANALISIS RACE CONDITION DAN SINKRONISASI

Tema: Server Layanan Akademik Kampus

Nama: Kafka Aditia
NIM: 250660121064
Kelas: IF-IIIA
Mata Kuliah: Sistem Operasi (USA-WP2260110)
Dosen: Muhammad Agreindra Helmiawan, S.Kom., M.T.

Universitas Sebelas April

Daftar Isi

1. Pendahuluan

2. Dasar Teori

3. Rancangan Program

4. Hasil Pengujian

5. Analisis Critical Section

6. Kesimpulan

7. Pernyataan Penggunaan AI

Lampiran A. Kode sebelum perbaikan

Lampiran B. Kode setelah perbaikan

Lampiran C. Screenshot

1. Pendahuluan

Tugas I-4 meminta program multithread yang menunjukkan race condition pada kasus dua mahasiswa memperebutkan satu kursi terakhir saat KRS, memperbaikinya dengan mutex atau semaphore, lalu membandingkan hasil serta menjelaskan critical section.

Tujuan praktik ini adalah menunjukkan akibat akses bersamaan pada data bersama dan menerapkan sinkronisasi agar jumlah kursi tetap konsisten.

2. Dasar Teori

Race condition terjadi ketika hasil program bergantung pada urutan waktu thread yang mengakses data bersama. Dua thread dapat sama-sama membaca kursi_tersedia = 1 sebelum salah satunya mengurangi nilai tersebut, sehingga keduanya mengaku berhasil. Mutex membatasi akses ke bagian kode penting sehingga satu thread pada satu waktu dapat memeriksa dan mengubah data.

3. Rancangan Program

Kedua program membuat dua thread, Mahasiswa A dan Mahasiswa B, dan memulai kursi_tersedia dengan nilai 1. Versi awal memakai Barrier untuk menyamakan waktu pemeriksaan, tetapi tidak mengunci perubahan kursi. Versi perbaikan menggunakan threading.Lock pada pemeriksaan, keputusan, dan pengurangan kursi.

4. Hasil Pengujian

4.1 Sebelum perbaikan

Contoh output dari satu eksekusi program:

```
Mahasiswa A: membaca kursi = 1
Mahasiswa B: membaca kursi = 1
Mahasiswa B: BERHASIL mengambil kursi
Mahasiswa A: BERHASIL mengambil kursi
Jumlah kursi akhir: -1
```

Pada versi ini kedua thread diarahkan membaca nilai kursi yang sama sebelum mengurangi nilainya. Output dapat menunjukkan dua keberhasilan dan jumlah kursi akhir -1. Urutan baris dapat berbeda.

4.2 Setelah perbaikan

Contoh output dari satu eksekusi program:

```
Mahasiswa A: memeriksa kursi = 1
Mahasiswa A: BERHASIL mengambil kursi
Mahasiswa B: memeriksa kursi = 0
Mahasiswa B: GAGAL, kursi sudah habis
Jumlah kursi akhir: 0
```

Mutex membuat pemeriksaan dan pengurangan kursi menjadi satu critical section. Thread pertama mengambil kursi; thread kedua menunggu lock dan kemudian mendapati kursi habis. Jumlah kursi akhir tetap 0.

Aspek	Sebelum	Sesudah
Penguncian	Tidak ada mutex pada operasi kursi	Pemeriksaan dan pengurangan dikunci
Hasil	Dua keberhasilan dapat terjadi	Maksimal satu keberhasilan
Jumlah akhir	Dapat menjadi -1	Tetap 0

Pengujian berulang: jalankan kedua program minimal lima kali. Ambil screenshot terminal dari eksekusi nyata pada perangkat sendiri. Contoh output di laporan ini bukan pengganti bukti screenshot.

5. Analisis Critical Section

Critical section adalah bagian yang memeriksa nilai kursi_tersedia, menentukan apakah kursi masih tersedia, dan mengurangi jumlah kursi bila pemesanan berhasil. Seluruh operasi ini harus berada di dalam mutex; mengunci hanya pengurangan tanpa mengunci pemeriksaan tetap dapat menimbulkan race condition.

6. Kesimpulan

Tanpa sinkronisasi, dua thread dapat mengklaim satu kursi dan menghasilkan nilai kursi yang tidak valid. Dengan mutex, hanya satu thread berhasil mengambil kursi dan jumlah akhir tetap nol.

7. Pernyataan Penggunaan AI

AI generatif digunakan untuk membantu membuat kerangka program, penjelasan konsep, dan draf laporan. Mahasiswa harus memahami dan memverifikasi kode, menjalankan pengujian sendiri, serta melampirkan screenshot nyata.

Lampiran A. Kode sebelum perbaikan

```
import threading, time
kursi_tersedia = 1
barrier = threading.Barrier(2)
lock_output = threading.Lock()

def mahasiswa(nama):
    global kursi_tersedia
    with lock_output:
        print(f"{nama}: membaca kursi = {kursi_tersedia}")
    if kursi_tersedia > 0:
        barrier.wait()       # Sengaja membuat kedua thread membaca nilai yang sama
        time.sleep(0.05)
        kursi_tersedia -= 1  # Tidak dilindungi mutex: race condition
        with lock_output:
            print(f"{nama}: BERHASIL mengambil kursi")

if __name__ == "__main__":
    ts = [threading.Thread(target=mahasiswa, args=(n,)) for n in ("Mahasiswa A","Mahasiswa B")]
    [t.start() for t in ts]
    [t.join() for t in ts]
    print("Jumlah kursi akhir:", kursi_tersedia)
```

Lampiran B. Kode setelah perbaikan

```
import threading, time
kursi_tersedia = 1
mutex = threading.Lock()

def mahasiswa(nama):
    global kursi_tersedia
    # Critical section: periksa dan ubah jumlah kursi di dalam mutex.
    with mutex:
        print(f"{nama}: memeriksa kursi = {kursi_tersedia}")
        if kursi_tersedia > 0:
            time.sleep(0.05)
            kursi_tersedia -= 1
            print(f"{nama}: BERHASIL mengambil kursi")
        else:
            print(f"{nama}: GAGAL, kursi sudah habis")

if __name__ == "__main__":
    ts = [threading.Thread(target=mahasiswa, args=(n,)) for n in ("Mahasiswa A","Mahasiswa B")]
    [t.start() for t in ts]
    [t.join() for t in ts]
    print("Jumlah kursi akhir:", kursi_tersedia)
```

Lampiran C. Bukti Eksekusi

Tempelkan screenshot terminal saat menjalankan race_condition.py dan synchronized.py, masing-masing minimal beberapa kali. Pastikan perintah dan hasil akhir jumlah kursi terlihat.